claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a12a854c83b57ed46.jsonl`
- SHA-256: `f631377dfe4797fe5a18885677265e6e23771cd020ece1866e66021b32f52427`
- Source modified: `2026-06-13T19:17:35+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:15:34.414Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"test_person_cue_adds_features_when_enabled asserts only key-presence; the feature VALUES and the real _person_features wiring are never validated","severity":"med","file":"tests/test_fusion_hybrid.py:98-109","detail":"The test returns ONE canned detections dict (frames {30,31}) for every window and only asserts the PERSON_FEATURE_NAMES keys exist. Because the same mock fires for both windows, both get identical features, and (I verified) shuttle_player_colocation computes to 0.0 for both ? the shuttle at frames 30/31 is at (500,100)/(500,900), nowhere near box (10,10,30,40). So the test passes whether or not _person_features wires anything correctly. None of the real glue is checked: the frame-range slice `shuttle_in = [p for p in points if start_f <= p.frame <= end_f]`, the court_polygon parse (`[(float(p[0]),float(p[1])) for p in poly]`), the net tuple `(net_axis, net_position)` construction, or the fw/fh fallback. A bug in any of those (e.g. swapping start_f/end_f, or never passing the polygon) would not fail this test.","fix":"Make detections_per_frame return per-window-distinct frames (side_effect keyed on start_frame), and assert at least one numeric feature value matches an independently-computed expectation (e.g. a window where shuttle sits in a person box yields shuttle_player_colocation>0, two_sided_motion>0 with players on both net halves). Also assert the polygon/net config actually reaches compute_person_features (e.g. via spy on ff.compute_person_features kwargs)."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:36.282Z)

I'll verify this finding by reading the actual test file and the relevant source code.

3. user (2026-06-13T19:15:37.235Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
```

```
      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
      persistence keeps the superset; and the person cue, when enabled, adds the person
features.
      """
      from types import SimpleNamespace
      from unittest.mock import MagicMock, patch

      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}


      def _points():
          pts = []
          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
          return pts


      def _runner():
          r = MagicMock()
          r.healthcheck.return_value = (True, "env ok")
          r.run_predict.return_value = "out.csv"
          r.parse_trajectory_csv.return_value = _points()
          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
          return r


      def _run(indexing=None, provider_segments=None):
          provider = MagicMock()
          provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
          db = MagicMock()
          db.add_candidate_segment.side_effect = [101, 102, 103, 104]
          db.add_play_segment.return_value = 200

          with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
               patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
               patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
               patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
               patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
               patch("os.environ.get", return_value="dummy_key"):
              preg.get.return_value = provider
              seg = FusionSegmenter(db, {"indexing": indexing or {}})
              res = seg.process_video("v1", "clip.mp4")
              return db, provider, res
```

```
57
58
59      def _stats_calls(db):
60          """Persisted stats dicts, in call order."""
61          return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64      def test_net_crossings_persisted_and_person_off_by_default():
65          db, _provider, _res = _run()
66          stats = _stats_calls(db)
67          assert len(stats) == 2                              # both windows persisted
(superset)
68          assert all("net_crossings" in s for s in stats)
69          # Window A (rally) crosses many times; window B (held) zero.
70          assert stats[0]["net_crossings"] >= 2
71          assert stats[1]["net_crossings"] == 0.0
72          # Person cue OFF by default -> NONE of the person features present.
73          assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74          # The 4 base motion keys are still there.
75          assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79      def test_persisted_candidates_are_superset_of_substrate_windows():
80          db, _provider, _res = _run()
81          spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82          assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered
83
84
85      def test_default_hands_off_all_windows():
86          _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
87          assert provider.analyze_video.call_count == 2
88
89
90      def test_gate_a_drops_low_crossing_window_from_handoff_only():
91          db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92          # Persistence is still the full superset (offline substrate intact)...
93          assert len(_stats_calls(db)) == 2
94          # ...but only the high-crossing window A reaches the AI handoff.
95          assert provider.analyze_video.call_count == 1
96
97
98      def test_person_cue_adds_features_when_enabled():
99          fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106         stats = _stats_calls(db)
107         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108         assert all("net_crossings" in s for s in stats)
109         fake_pd.detections_per_frame.assert_called()         # the person detector WAS used
110
```

4. user (2026-06-13T19:15:37.721Z)

```
1       """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
```

```
2
3       NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4       math is unit-testable in a GPU-free dev container. The detection (person boxes,
5       shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6       module only turns those signals into a handful of **scale/translation-invariant**
7       features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8       memorise this court/camera and fail to generalise off a small labelled set).
9
10      Conventions:
11      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
12      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21      """
22      from __future__ import annotations
23
24      from typing import Dict, List, Optional, Sequence, Tuple
25
26      BBox = Tuple[float, float, float, float]
27      Net = Tuple[str, float]
28
29
30      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33          if polygon is None or len(polygon) < 3:
34              return False
35          x, y = point
36          inside = False
37          n = len(polygon)
38          j = n - 1
39          for i in range(n):
40              xi, yi = polygon[i]
41              xj, yj = polygon[j]
42              # Does the horizontal ray at y cross edge (i, j)?
43              if (yi > y) != (yj > y):
44                  x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                  if x < x_cross:
46                      inside = not inside
47              j = i
48          return inside
49
50
51      def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52          """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53          the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54          if frame_width <= 0 or frame_height <= 0:
55              return (0.0, 0.0)
56          x1, _y1, x2, y2 = box
57          return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
```

```

    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None."""
        if court_polygon is None:
            return True
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, normalised
        by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
```

```
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
        width/height (the 'adjacent / in-hand' tolerance)?"""
        x1, y1, x2, y2 = box
        px = pad_frac * frame_width
        py = pad_frac * frame_height
        return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)


    def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
                                  frame_width: float, frame_height: float,
                                  pad_frac: float = 0.03) -> float:
        """Fraction of (visible shuttle frames that were also person-sampled) in which the
        shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
        ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
        """
        if not per_frame or frame_width <= 0 or frame_height <= 0:
            return 0.0
        aligned = 0
        colocated = 0
        for p in shuttle_points or []:
            if not getattr(p, "visible", False):
                continue
            dets = per_frame.get(int(p.frame))
            if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
                continue
```

```
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                    for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                 frame_width: float, frame_height: float,
198                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                 net: Optional[Net] = None,
200                                 min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212
```

5. assistant (2026-06-13T19:15:41.958Z)

Now let me look at the actual wiring in `_person_features` in the fusion segmenter to verify the claims in the finding.

6. user (2026-06-13T19:15:42.456Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
```

```
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                      "AI provider sets semantic boundaries.")
57
58          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60              if substrate == "tracknet":
61                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62                  return TrackNetRunner(cfg), {
63                      "weights_path": cfg.weights_path,
64                      "queue_length": cfg.queue_length,
65                      "fusion_substrate": "tracknet",
66                  }
67              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                      frame_width: float, video_path: str,
72                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
```

```
video_path, idx_cfg)
74              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76              # over the whole trajectory by rally_gate (camera-agnostic).
77              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80              fcfg = idx_cfg.get("fusion", {})
81              if fcfg.get("cue_flags", {}).get("person", False):
82                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83              return stats
84
85          def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                               frame_width: float, video_path: str,
87                               fcfg: Dict[str, Any]) -> Dict[str, float]:
88              """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89              frame range and compute the scale/translation-invariant person features. Lazily
builds
90              ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91              # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92              from backend.pipeline.detectors.person_detector import PersonDetector
93              if self._person is None:
94                  self._person = PersonDetector(self.config)
95
96              frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97              start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98              det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99              fw = det["frame_width"] or frame_width
100             fh = det["frame_height"] or 0.0
101
102             poly = fcfg.get("court_polygon")
103             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                               court_polygon=polygon, net=net)
108
109         def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                                 window_stats: List[Dict[str, Any]],
111                                 idx_cfg: Dict[str, Any]) -> List[int]:
112             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
```

```
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

7. assistant (2026-06-13T19:15:49.549Z)

Now I understand the actual wiring. Let me verify the finding's specific claims about the test. The key claims are:

1. The mock returns ONE canned detections dict (frames {30,31}) for every window.
2. `shuttle_player_colocation` computes to 0.0 for both windows because the shuttle at frames 30/31 is at (500,100)/(500,900), nowhere near box (10,10,30,40).

Let me verify the colocation math precisely. The mock has `frame_width=1000.0`, `frame_height=1000.0`, `pad_frac=0.03`. So `px = py = 0.03 * 1000 = 30`.

Box `(10,10,30,40)`: padded x-range = `[-20, 60]`, padded y-range = `[-20, 70]`.

Shuttle at frame 30: `x=500, y=100` (even frame ? y=100). Box for frame 30: `(10,10,30,40)`. Shuttle x=500 is NOT in [-20,60]. So not colocated.

Let me verify the frame-range slice claim and check whether the test would catch bugs. Let me trace `shuttle_in` and which shuttle points align with the per_frame dict.

8. user (2026-06-13T19:15:50.840Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
37      def _fmt_ts(seconds: float) -> str:
38          seconds = max(0, int(seconds))
```

```
39          h, rem = divmod(seconds, 3600)
40          m, s = divmod(rem, 60)
41          return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44      class TrajectoryHybridSegmenter(BasePlaySegmenter):
45          """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""
46
47          #: config section under `indexing` (velocity_threshold lives there), the
48          #: output subdir for trajectories, and the metadata detector-tag prefix.
49          family: str = ""
50          #: human-readable label used in log lines.
51          display_label: str = ""
52
53          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
54              """Return (runner, detector-specific detection_params extras)."""
55              raise NotImplementedError
56
57          @staticmethod
58          def _probe_fps_width(video_path: str) -> tuple:
59              """Return (fps, frame_width) for a video, with sensible fallbacks."""
60              cap = cv2.VideoCapture(video_path)
```

9. assistant (2026-06-13T19:15:55.076Z)

Now let me trace the actual call to verify the finding's central claim. The key is what `_enrich_candidate_stats` receives as `cw`, `points`, `fps`. Let me find where it's called in the base class.

10. user (2026-06-13T19:15:56.865Z)

```
79-             return max(3, int(duration / 0.8))
80-
81-     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
82:     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83-                                 frame_width: float, video_path: str,
84-                                 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85-         """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
--
93-             "track_id_count": cw["track_id_count"],
94-         }
95-
96:     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97-                             window_stats: List[Dict[str, Any]],
98-                             idx_cfg: Dict[str, Any]) -> List[int]:
99-         """Indices of the candidate windows that proceed to the AI handoff (and thus may
--
201-         # segmenter adds net_crossings + person-cue features. Computed once, reused for both
202-         # persistence and the handoff gate below.
203-         window_stats = [
204:             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
205-             for cw in all_candidate_windows
206-         ]
207-
--
222-         # --- Phase 3: AI provider handoff over padded candidate clips ---
223-         # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
224-         # candidates above stay the full superset ? only the served play_segments are gated.
225:         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226-         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227-         os.makedirs(handoff_dir, exist_ok=True)
228-         logger.info("Phase 3: %s validation on %d candidate windows...",
```

11. assistant (2026-06-13T19:16:06.252Z)

Now I have the full picture. Let me verify the central claim by tracing the test precisely.

Key facts established:
- `_person_features` is called once per candidate window (windows A and B), both with the SAME `points` list (the full trajectory from `_points()`), and the SAME mock `det` dict (the mock `fake_pd.detections_per_frame` returns the same value every call ? no `side_effect`).
- `det["frames"]` = `{30: [(1,(10,10,30,40))], 31: [(1,(12,10,32,40))]}`, `fw=1000`, `fh=1000`.

Now let me precisely verify each of the 5 person features for both windows to determine whether the test would catch real bugs.

Window A: `cw = WIN_A`, `start=1.0, end=4.0`, `fps=30`. So `start_f=30, end_f=120`.
`shuttle_in = [p for p in points if 30 <= p.frame <= 120]` ? frames 30..119 from window-A points (the 300-359 points are excluded).

Window B: `cw = WIN_B`, `start=10.0, end=12.0`, `fps=30`. So `start_f=300, end_f=360`.
`shuttle_in = [p for p in points if 300 <= p.frame <= 360]` ? frames 300..359.

Now the `det["frames"]` dict is `{30, 31}` for BOTH windows.

Let me compute each feature carefully.

12. user (2026-06-13T19:16:06.882Z)

```
82          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83                                      frame_width: float, video_path: str,
84                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85              """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
86              returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87              rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88              features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89              return {
90                  "peak_velocity": cw["peak_velocity"],
91                  "mean_velocity": cw["mean_velocity"],
92                  "active_frame_count": cw["active_frame_count"],
93                  "track_id_count": cw["track_id_count"],
94              }
95
96          def _select_for_handoff(self, windows: List[Dict[str, Any]],
```

13. assistant (2026-06-13T19:16:23.468Z)

Now let me compute the features precisely for each window to determine whether the test discriminates.

Common `det["frames"]` = `{30: [(1,(10,10,30,40))], 31: [(1,(12,10,32,40))]}`, fw=fh=1000, court_polygon=None (default fcfg has no `court_polygon`), net=("y", 0.5).

Let me compute:

1. `players_in_court`: court_polygon=None ? `person_in_court` always True. counts = [1, 1] (one person each frame). median = 1.0. **Same (1.0) for both windows.**

2. `in_court_ratio` (min_players default 2): frames with >=2 in court = 0. ratio = 0/2 = 0.0. **Same (0.0) for both windows.**

3. `mean_player_motion`: 2 frames {30,31}, track 1 in both. center30 = ((10+30)/2/1000, (10+40)/2/1000) = (0.02, 0.025). center31 = ((12+32)/2/1000, (10+40)/2/1000) = (0.022, 0.025). displacement = 0.002. **Same (0.002) for both windows.**

4. `two_sided_motion`: net=("y",0.5). For each frame, foot_norm y. Box1 foot y = 40/1000=0.04 < 0.5 ? near. Box2 foot y=40/1000=0.04 ? near. Both frames only have "near", never "far". both=0. result=0/2=0.0. **Same (0.0) for both windows.**

5. `shuttle_player_colocation`:
- Window A: `shuttle_in` includes frames 30,31 (among 30..119). For frame 30: visible, dets exist ? aligned. shuttle x=500,y=100. Box (10,10,30,40) padded ±30 ? x in [-20,60]. 500 not in range ? not colocated. Frame 31: x=500,y=900, box (12,10,32,40) padded ? x in [-18,62]. Not colocated. Other shuttle frames 32..119 have no dets in per_frame ? skipped. aligned=2, colocated=0 ? 0.0.
- Window B: `shuttle_in` = frames 300..359. None of these are in per_frame {30,31} ? `dets = per_frame.get(frame)` is None for all ? aligned=0 ? returns 0.0.

So **colocation = 0.0 for both windows** ? confirmed as the finding states.

Now let me verify the finding's claim that ALL features are identical / non-discriminating. Let me double check the `players_in_court` value, which is NOT 0.0 ? it's 1.0 for both. This is a non-trivial numeric value. Let me verify the test assertion precisely. The test asserts:

```python
assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
```

This only checks key-presence ? confirmed. It does NOT assert any value.

Let me now adversarially test the finding's core claim: would a real bug in the glue go undetected? Let me check the specific bugs the finding enumerates.

14. user (2026-06-13T19:16:24.278Z)

```
1       """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3       Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4       the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5       `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6       next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7       later the learned classifier reads its persisted features. Keeping detection here (not
in
8       the segmenter) is what makes "add a cue = register a producer" true.
9
10      Behaviour is unchanged from the in-lined version ? only the home moved:
11        - `lazy_load_model`               ? load a permissively-licensed torchvision detector
12        - `make_tracker`                  ? a fresh supervision ByteTrack per chunk
13        - `detect_and_track`              ? one frame ? confident persons with persistent IDs
14        - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15          with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16          that feed the candidate `stats` JSON and the learned fusion features.
17
18      Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19      the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20      the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21      (ADR-005; ultralytics' AGPL YOLO was dropped).
22      """
23      import logging
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27      import torch
28
29      from backend.utils.geometry import get_velocity_normalised
30
31      logger = logging.getLogger(__name__)
32
```

```
33      # torchvision detection models are trained on COCO where the "person" category is
34      # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35      # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36      _PERSON_CLASS_ID = 1
37
38      # Permissively-licensed torchvision detectors selectable via config
39      # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40      _DETECTOR_FACTORIES = {
```

15. assistant (2026-06-13T19:16:27.770Z)

Now let me find the `detections_per_frame` signature to confirm the mock matches the real method signature, which is relevant to whether the test even exercises the right call.

16. user (2026-06-13T19:16:29.140Z)

```
256:    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257-                             frame_skip: int = 3) -> Dict[str, Any]:
258-        """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
video.
259-
260-        Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
261-        ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards
the
262-        per-frame boxes), this returns the raw per-frame detections so the fusion features
can
263-        align them with the shuttle trajectory. It reads the source by **absolute frame
index**
264-        (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
match
265-        the WASB trajectory's ``.frame`` exactly ? that alignment is what
266-        ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
267-
268-        Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
269-        "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
270-        accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
271-        treat the indices as ±a few frames on real footage ? fine for window-level features.
272-        """
273-        self.lazy_load_model()
274-        cap = cv2.VideoCapture(video_path)
275-        if not cap.isOpened():
276-            logger.error(f"Could not open {video_path}")
277-            return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
278-
279-        fps = cap.get(cv2.CAP_PROP_FPS)
280-        if fps <= 0:
281-            fps = 30.0
```

17. assistant (2026-06-13T19:16:46.336Z)

The mock matches the real signature and return shape. Now I have everything I need to adjudicate the finding.

Let me confirm the central claim about whether the test would catch the enumerated glue bugs. Let me re-examine the patch target. The test patches `backend.pipeline.detectors.person_detector.PersonDetector`, but `_person_features` does `from backend.pipeline.detectors.person_detector import PersonDetector` lazily inside the function ? so patching the source module attribute works. Good, the mock IS hit (`assert_called` passes).

Now, the finding's core thesis: **the test asserts only key-presence, never values, and the same canned mock fires for both windows, so all 5 features are non-discriminating, meaning glue bugs would not be caught.**

Let me adversarially probe each enumerated glue bug to see if it's truly uncaught:

Bug 1: swapping `start_f`/`end_f` ? `shuttle_in = [p for p in points if end_f <= p.frame <= start_f]`. For window A: `120 <= frame <= 30` ? empty list. But colocation uses `per_frame` keys for alignment, and the shuttle_in being empty ? colocation iterates over empty ? 0.0. Still 0.0. The other 4 features don't use `shuttle_in` at all. So **swapping start_f/end_f ? no change in any asserted value ? uncaught.** Confirmed.

Bug 2: never passing the polygon ? `compute_person_features(court_polygon=None)`. With court_polygon=None, `person_in_court` returns True (same as the test, since fcfg has no `court_polygon` key ? polygon=None already). So in THIS test the polygon is already None. The test cannot distinguish polygon-passed vs not. Confirmed uncaught ? but note this is partly because the test config doesn't set a polygon at all.

Bug 3: net tuple construction ? if net were never passed (None), `two_sided_motion` returns 0.0. In the test, with net=("y",0.5), two_sided_motion is ALSO 0.0 (both players near). So **net=None vs net=("y",0.5) both yield 0.0 ? uncaught.** Confirmed.

Bug 4: fw/fh fallback ? `fw = det["frame_width"] or frame_width`. The mock sets frame_width=1000.0 (truthy), so the fallback branch isn't exercised. Confirmed the fallback path is untested.

So the finding's technical claims all hold. The test genuinely only validates key-presence, and every feature value is non-discriminating (colocation=0, two_sided=0, in_court_ratio=0, players_in_court=1 constant, motion=0.002 constant). A bug in the frame-range slice, polygon parse, or net construction would not fail this test.

Now let me weigh severity. This is a test-quality / coverage gap, not a production bug. The finding itself rates it "med." Let me consider counter-arguments to be fair:

- Other tests in the file DO validate `net_crossings` values discriminatively (window A >= 2, window B == 0), so the net-crossing Gate A glue IS well-tested.
- The person-feature math itself is presumably unit-tested in a separate `fusion_features` test file. Let me check that, because if the pure math is thoroughly tested elsewhere, the finding's scope narrows to just the *wiring/glue* (which is exactly what the finding claims is untested).

18. user (2026-06-13T19:16:46.546Z)

```
tests\test_fusion_features.py
tests\__pycache__\test_fusion_features.cpython-313-pytest-9.0.3.pyc
tests\test_fusion_hybrid.py
tests\__pycache__\test_fusion_hybrid.cpython-313-pytest-9.0.3.pyc
```

19. user (2026-06-13T19:16:50.467Z)

```
24:def test_point_in_polygon_inside_outside():
31:def test_point_in_polygon_degenerate():
38:def test_person_in_court_uses_foot_point_and_polygon():
52:def test_in_court_counts_and_players_and_ratio():
70:def test_mean_player_motion():
79:# ------------------------------------------------------------------ two_sided_motion
81:def test_two_sided_motion():
86:    assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
88:    assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89:    assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0   # no net -> 0
92:# ------------------------------------------------------- shuttle_player_colocation
94:def test_shuttle_player_colocation():
100:    assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) == pytest.approx(0.5)
103:    assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
105:    assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106:    assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100, 100)
== 0.0
111:def test_compute_person_features_keys():
116:    feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
120:    assert feats["two_sided_motion"] == pytest.approx(1.0)   # both sides occupied every
```

```
frame
123:def test_compute_person_features_empty():
124:    feats = ff.compute_person_features({}, [], 100, 100)
```

20. assistant (2026-06-13T19:16:58.456Z)

This confirms the picture precisely. The **pure feature math is thoroughly unit-tested** in `test_fusion_features.py` (colocation discriminates 0.5 vs 1.0, two_sided 0.5 vs 0.0, etc., with independently-computed expectations). So the math itself is well-covered.

That means `test_person_cue_adds_features_when_enabled` in `test_fusion_hybrid.py` is the ONLY test covering the **segmenter-level glue** in `_person_features` (lines 85-107): the frame-range slice, polygon parse, net-tuple construction, fw/fh fallback, and passing `det["frames"]`/`shuttle_in` correctly into `compute_person_features`. And as established, that test only asserts key-presence and uses a non-discriminating canned mock, so none of that glue is value-validated.

This validates the finding accurately. Let me make one final fairness check on the finding's specific factual claims about the colocation math and the "(500,100)/(500,900)" assertion, which I already computed match. The finding is technically precise.

Let me also verify there isn't some OTHER test that exercises the glue with discriminating values (e.g., a polygon-passing test elsewhere).

21. user (2026-06-13T19:16:59.422Z)

```
Found 4 files
tests\test_config.py
tests\test_fusion_hybrid.py
tests\test_fusion_features.py
tests\test_experiment.py
```

22. user (2026-06-13T19:17:03.031Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_config.py:33:    assert
f.cue_flags == {}            # person cue OFF
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_config.py:36:    assert
f.court_polygon is None      # no mask
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_config.py:46:
"cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1], [0.5, 0.9]],
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_config.py:52:        assert
f.cue_flags == {"person": True}
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_config.py:53:        assert
len(f.court_polygon) == 3
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_experiment.py:58:    v =
Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_experiment.py:81:    disabled =
Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
```

23. assistant (2026-06-13T19:17:08.950Z)

`test_config.py` validates the config *parsing* (`court_polygon` parsed into the model), but that is config-layer, not the segmenter glue that reads `fcfg.get("court_polygon")` and passes it through `_person_features` ? `compute_person_features`. No other test exercises the `_person_features` glue with discriminating values. The finding holds.

Let me finalize my assessment.

The finding is **REAL** and accurately described. Severity: I agree with **med** ? it is a genuine test-coverage gap, not a false positive or nitpick, but it is a test-quality issue rather than a production bug, and the surrounding tests + the dedicated `test_fusion_features.py` mitigate the risk that the underlying feature math is wrong. The uncovered surface is specifically the segmenter-level wiring.

24. user (2026-06-13T19:17:27.574Z)

Structured output provided successfully

25. assistant (2026-06-13T19:17:34.902Z)

I verified the finding against the actual code and confirmed it is REAL (severity: med).

Key files inspected:
- `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_fusion_hybrid.py:98-109` ? the test under review
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/fusion_hybrid.py:85-107` ? `_person_features` glue
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/fusion_features.py` ? pure feature math
- `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/trajectory_hybrid.py:203-206` ? per-window call site
- `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_fusion_features.py` and `tests/test_config.py` ? checked for compensating coverage

The finding's technical claims are precise: the canned mock fires identically for both windows, I computed all five person features and every one is non-discriminating (colocation=0, two_sided=0, in_court_ratio=0, players_in_court=1 constant, mean_player_motion=0.002 constant), and the test asserts only key-presence. I confirmed each enumerated glue bug (start_f/end_f swap, missing polygon, net tuple, fw/fh fallback) would pass undetected. The math is covered by `test_fusion_features.py`, so the gap is specifically the segmenter-level wiring ? which this is the only test for.